Graficadora de Transformación de Funciones

Proyecto Final · Curso de Precálculo

Guion de Exposición

$$g(x) = a \cdot f(\, b\,(x - c)\,) + d$$

Cómo usar este guion

Cada bloque es una **diapositiva / momento de la exposición**. El recuadro *"Qué decir"* es el libreto hablado: léelo con naturalidad, no de memoria. La meta: que el jurado entienda **qué construimos**, **por qué funciona** y **qué lo hace distinto**. Tiempo sugerido total: **8–10 minutos**.

1 Apertura con gancho

Qué decir: *Buenos días. Todos aprendimos que mover una gráfica un poco a la derecha o estirarla cambia su fórmula… pero verlo en papel es lento y abstracto. Nosotros construimos una* ***graficadora interactiva*** *que toma cualquier función y, en tiempo real, muestra cómo se transforma. En vez de imaginarlo, lo vemos.*

2 El problema y el objetivo

- Entender las transformaciones de funciones es difícil cuando solo se ven en pizarra.
- Objetivo: una app que una la **matemática de precálculo** con la **programación** de nuestra carrera.

Qué decir: *El reto del proyecto era integrar lo que vemos en Precálculo con lo que programamos. Elegimos las transformaciones de funciones porque son visuales, y la programación es perfecta para volverlas interactivas.*

3 El modelo matemático central

$$g(x) = a \cdot f(b (x - c)) + d$$

- **a** → estira/comprime en vertical; si es negativo, refleja sobre el eje X.
- **b** → estira/comprime en horizontal; si es negativo, refleja sobre el eje Y.
- **c** → mueve la gráfica a la derecha o a la izquierda.
- **d** → mueve la gráfica hacia arriba o hacia abajo.

Qué decir: *Toda transformación cabe en esta sola ecuación. Cambiando cuatro números —a, b, c, d— podemos estirar, comprimir, reflejar y mover cualquier función. Nuestra app no hardcodea casos: implementa este modelo general y de ahí salen todos los efectos.*

4 Demostración en vivo (el momento clave)

- Abrir la app en el navegador (`python app.py` → `127.0.0.1:5000`).
- Elegir la **cuadrática** y poner `a=2, c=1, d=3` .
- Mostrar cómo la curva naranja (original) y la azul (transformada) se separan.
- Cambiar `a` a **negativo** y señalar la reflexión.

Qué decir: *Miren: esta es la parábola normal, punteada. Ahora pongo a=2 y la estira; c=1 la mueve a la derecha; d=3 la sube. Y si hago a negativo… se voltea. El resumen de abajo explica con palabras cada cambio, y la tabla muestra los valores exactos. Todo esto lo calcula Python en el instante.*

5 Cómo está hecho (arquitectura)

Python · Flask | HTML | CSS | JavaScript | Chart.js

- **Backend en Python:** es el "cerebro". Hace todos los cálculos matemáticos y la validación.
- **Frontend web:** la cara visible. Solo pide los números a Python y los dibuja.

Qué decir: *Decidimos separar el cerebro de la cara. El cerebro es Python: ahí vive toda la matemática. La cara es una página web. El navegador no calcula nada; le pregunta a Python y dibuja la respuesta. Así cumplimos "hecho en Python" y además es una web moderna.*

6 Código bien organizado (modular)

- `funciones.py` → las 6 funciones madre y sus dominios.
- `transformaciones.py` → el modelo g(x), la tabla y el resumen.
- `validacion.py` → revisa que las entradas sean correctas.

Qué decir: *No metimos todo en un solo archivo gigante. Separamos por responsabilidad: las funciones, las transformaciones y la validación viven aparte. Eso es lo que pedía el proyecto —lógica, datos y presentación desacoplados— y hace el código fácil de mantener.*

7 Lo que lo hace robusto

- Si escribes **letras** donde van números, usa valores por defecto y no truena.
- Respeta el **dominio**: no intenta la raíz de un número negativo.
- Valores por defecto: `a=1, b=1, c=0, d=0`.

Qué decir: *Pensamos en el usuario que se equivoca. Si escribe "hola" en vez de un número, la app no se cae: usa un valor por defecto. Y nunca intenta calcular la raíz de un negativo, porque validamos los dominios. Es matemáticamente correcto y a prueba de errores.*

8 Lo que entrega la app

- Gráfica doble: original **punteada** + transformada **continua**.
- Resumen **en español** de cada transformación.
- Tabla de puntos (x, f(x), g(x)).
- La fórmula de g(x) construida automáticamente.

> **Qué decir:** *La app no solo dibuja: explica. Da el gráfico, la descripción en palabras, la tabla de valores y la fórmula final. Un estudiante puede usarla para entender, no solo para ver un dibujo.*

9 Cierre e impacto

> **Qué decir:** *Convertimos un tema abstracto de Precálculo en una herramienta viva. Unimos matemática y programación en algo que de verdad ayuda a aprender. Está hecho en Python, es modular, es a prueba de errores y es una web que cualquiera puede abrir. Gracias — ¿preguntas?*

Posibles preguntas del jurado (y cómo responder)

"¿Por qué web si pedían Python?"

> **Qué decir:** *Todo el cálculo está en Python (Flask). La web es solo la interfaz; el navegador no hace matemática, se la pide a Python. Cumplimos ambas cosas.*

"¿Qué pasa si pongo mal un dato?"

> **Qué decir:** *La validación lo atrapa: usa valores por defecto o avisa con un mensaje, y nunca rompe el dominio de la función.*

"¿Podrían agregar más funciones?"

> **Qué decir:** *Sí, muy fácil: solo se añade una entrada en* `funciones.py` *con su fórmula y su dominio. El resto del sistema la toma automáticamente. Por eso separamos el código.*

Backend en Python (Flask) · Frontend en HTML / CSS / JavaScript

$$g(x) = a \cdot f(b (x - c)) + d$$